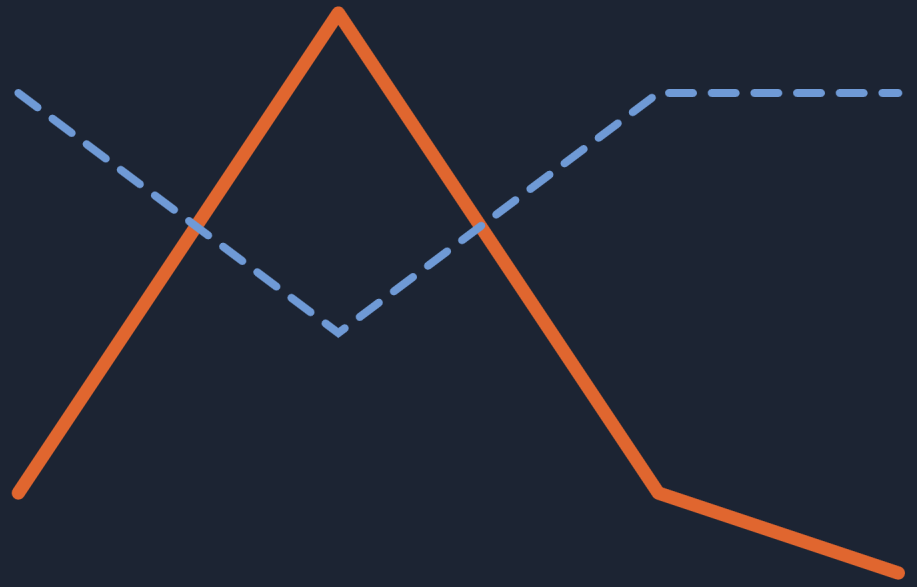
ЭФФЕКТИВНЫЕ МОДЕЛИ · ДЗ №2 · РАЗБОР СТАТЬИ

Super-Convergence

Очень быстрое обучение нейросетей с большим learning rate

Leslie N. Smith, Nicholay Topin · arXiv:1708.07120 (2017–2018)



Подготовил: Андрей, магистрант 1 курса · 2026

План доклада

01 Проблема: обычное обучение нейросетей идёт долго

02 Метод: циклический LR, LR range test и политика 1cycle

03 Почему это работает: большой LR как регуляризация

04 Эксперименты: CIFAR-10/100, MNIST, ImageNet

05 Моё воспроизведение: ResNet-56 на CIFAR-10

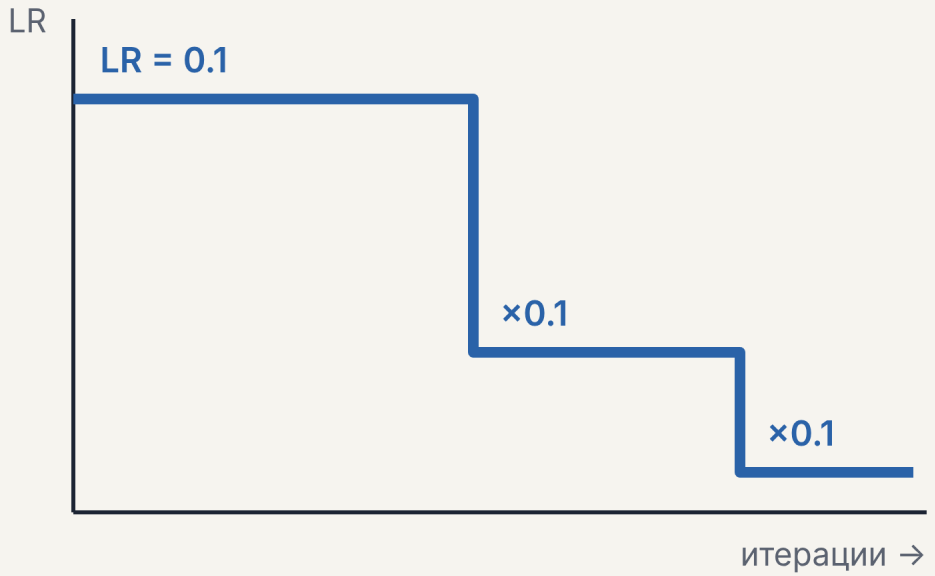
06 Выводы и критический взгляд

Как обычно учат нейросети

Стандартный режим — **кусочно-постоянный learning rate** (piecewise constant):

1. Учим с LR ≈ 0.1, пока точность не выйдет на плато
2. Уменьшаем LR в 10 раз и продолжаем
3. Повторяем ещё 2–3 раза

Итог: сотни эпох и много вычислений. Например, ResNet-56 на CIFAR-10 — **80 000 итераций**.



Вопрос статьи: можно ли учить на порядок быстрее — и не потерять в точности?

Что такое super-convergence

Явление, когда сеть с **очень большим LR** обучается на порядок быстрее и до более высокой точности.

92.4%
за 10 000 итераций · CLR, LR 0.1–3

91.2%
за 80 000 итераций · обычный LR = 0.35

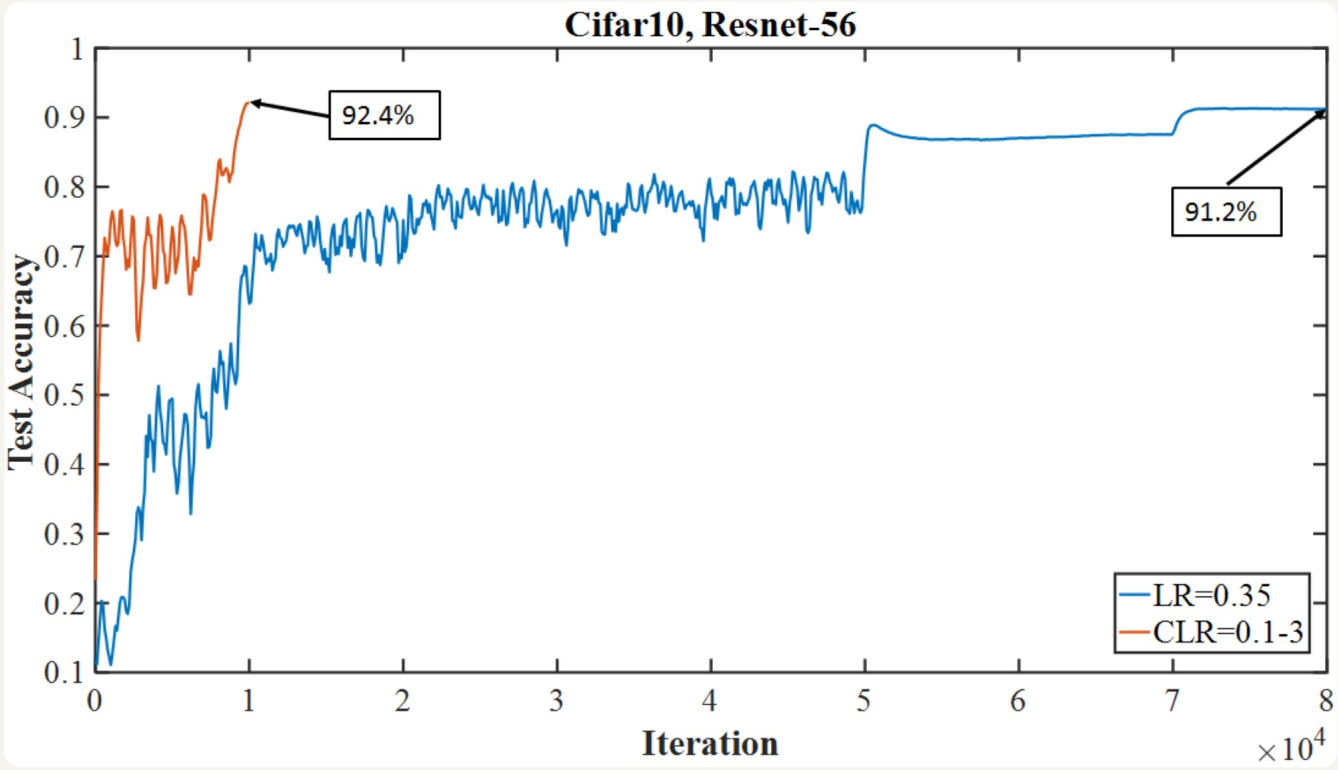
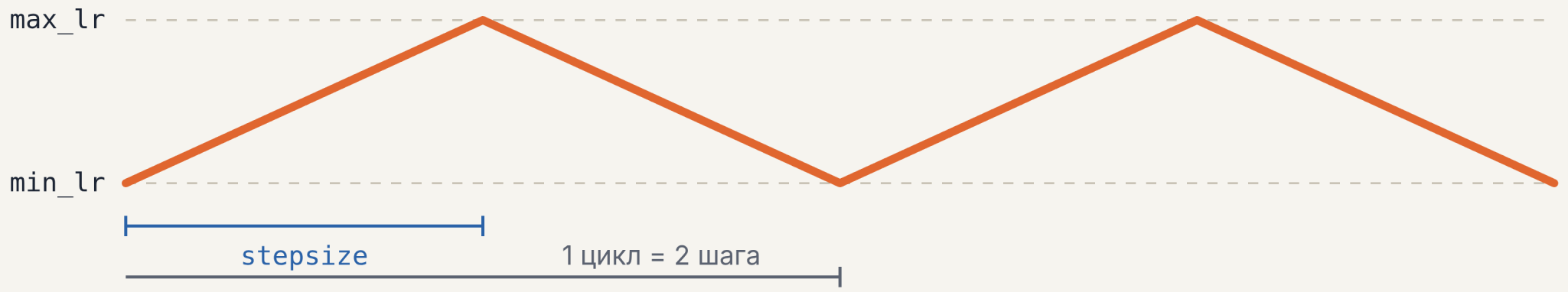


Рис. 1a из статьи: ResNet-56, CIFAR-10

Основа: циклический learning rate (CLR)



Границы

LR линейно ходит между min_lr и max_lr

Stepsize

Число итераций на подъём (или спуск)

Идея

Смесь curriculum learning и simulated annealing

Как выбрать границы: LR range test

1. Короткий прогон: LR линейно растёт от ~0 до большого значения
2. Строим точность на тесте в зависимости от LR
3. Пик кривой → max_lr, а min_lr ≈ max_lr / 3–4

Необычное наблюдение

У ResNet-56 на CIFAR-10 точность держится высокой даже при LR = 3 — на порядок больше обычного. Это сигнал, что super-convergence возможна.

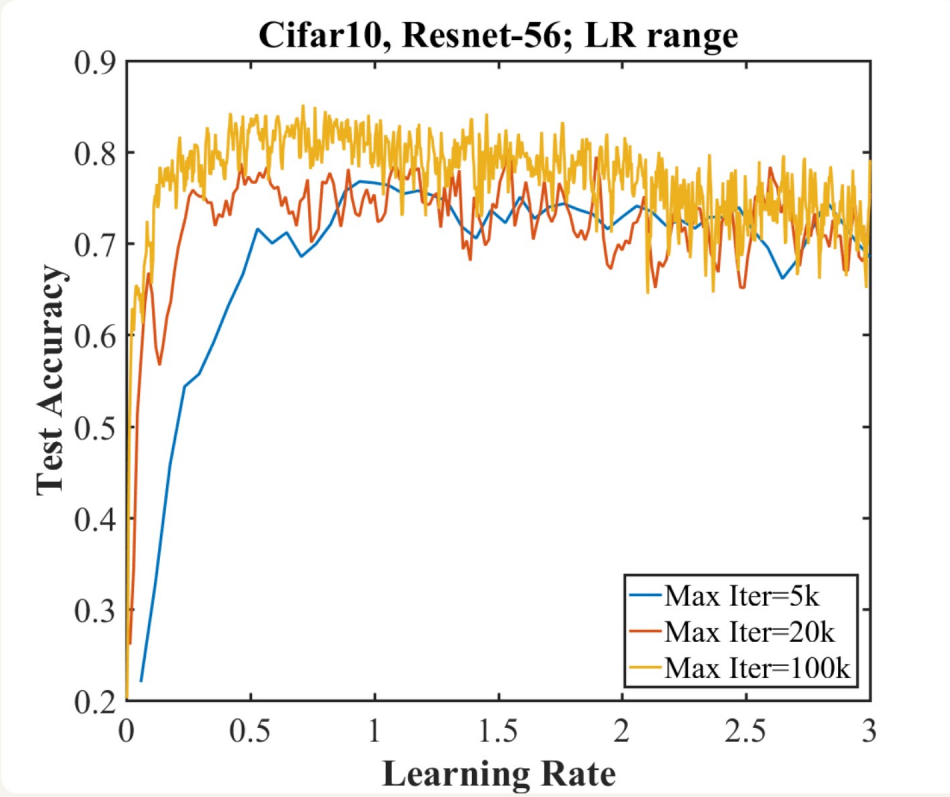
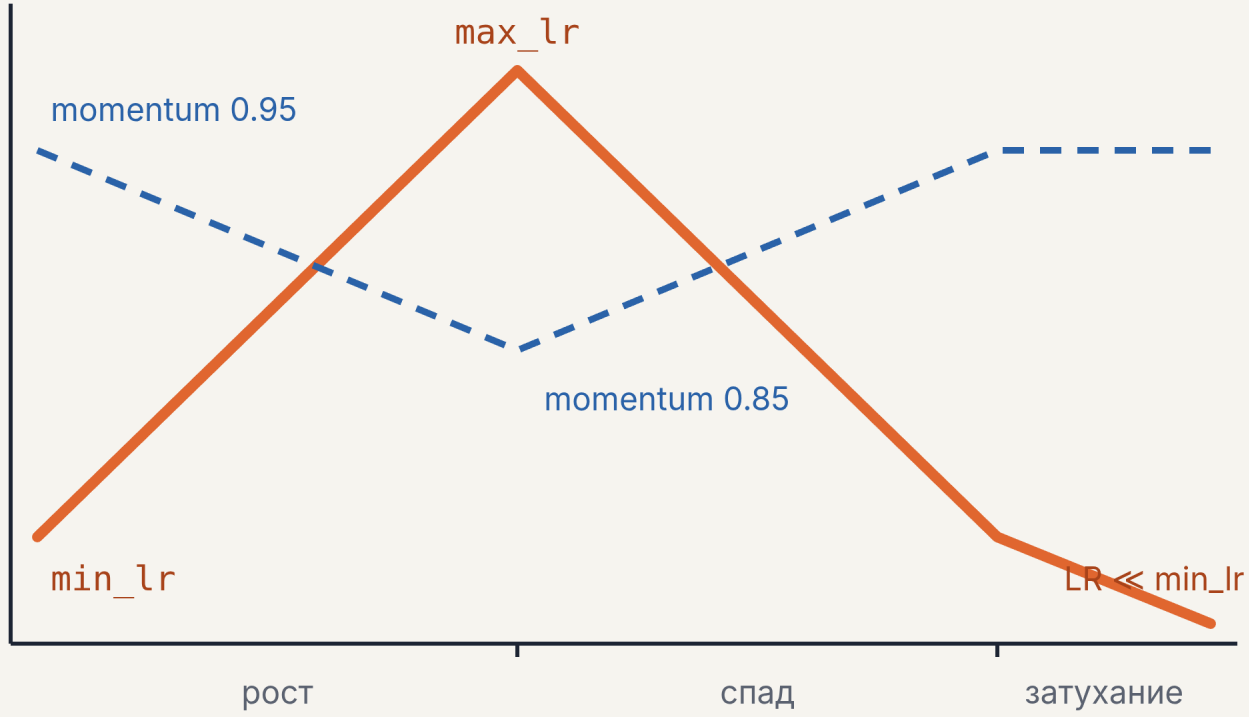


Рис. 2b из статьи

Политика 1cycle



Один цикл, чуть короче всего обучения

В конце LR опускается на несколько порядков ниже min_lr

Momentum меняется в противофазе: 0.95 → 0.85 → 0.95

Сегодня это готовый OneCycleLR в PyTorch

Ключевая идея: большой LR — это регуляризация

Увеличиваем ↑

Learning rate — до 1–3 вместо привычных 0.1

⇄

Уменьшаем ↓

Weight decay, dropout, «шум» маленького батча

Суммарная регуляризация должна быть сбалансирована под конкретные датасет и архитектуру.

Пример: на ImageNet weight decay снизили с 10^{-4} до $3 \cdot 10^{-6}$ – 10^{-5} — только так LR до 1.0 дал выигрыш.

Оценка оптимального LR через гессиан

Полный гессиан не нужен: важна кривизна только вдоль направления спуска. Её оценивают по разности градиентов:

$$\varepsilon^* \approx \Delta\theta \;/\; \Delta\nabla f$$

Подставим шаг SGD $\theta_{i+1} = \theta_i - \varepsilon\nabla f(\theta_i)$:

$$\varepsilon^* = \varepsilon \cdot (\theta_{i+1} - \theta_i) \;/\; (2\theta_{i+1} - \theta_i - \theta_{i+2})$$

Общий LR: сумма |числителей| / сумма |знаменателей| по всем весам

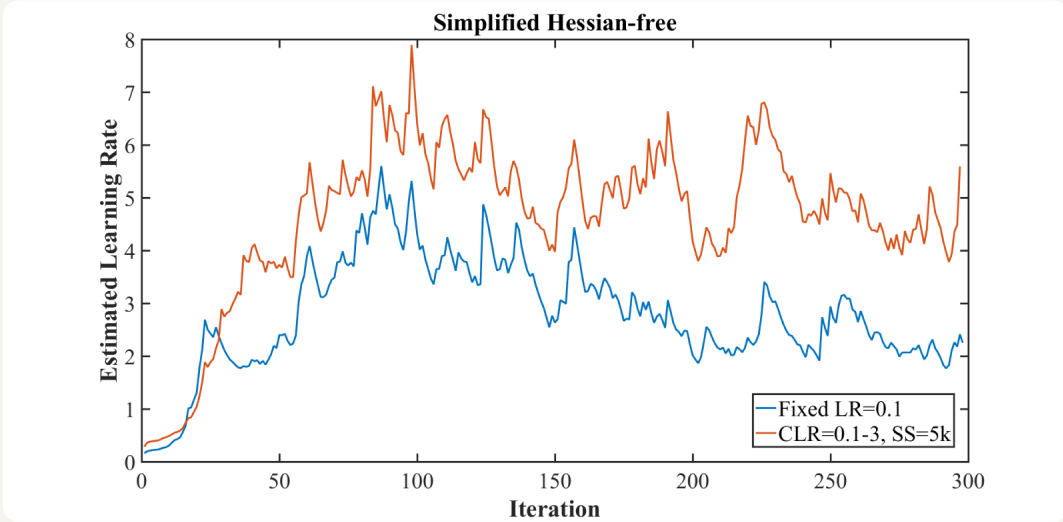


Рис. 3a: оценка ε* по ходу обучения ResNet-56

По мнению авторов: LR ≈ 2–6 ⇒ кривизна мала ⇒ SGD находит широкие плоские минимумы

Это интерпретация: сам метод оценки в статье полноценно не проверялся

Эксперимент: когда данных мало

Обучающих примеров	Обычный LR 80k итераций	CLR 0.1–3 10k итераций	Разница
40 000	89.1%	91.1%	+2.0
30 000	85.7%	89.6%	+3.9
20 000	82.7%	87.9%	+5.2
10 000	71.4%	80.6%	+9.2

+9.2

процентного пункта при 10 000 примеров — и в 8 раз меньше итераций

Чем меньше размеченных данных, тем сильнее выигрыш от super-convergence.

Табл. 1 статьи: ResNet-56, CIFAR-10 · в моём воспроизведении разрыв меньше — слайд 16

Работает на разных данных и архитектурах

Датасет · сеть	Обычный LR	1cycle
CIFAR-10 · Wide ResNet-32	90.3% за 800 эпох	91.9% за 100 эпох
CIFAR-10 · DenseNet-40	92.7% за 400 эпох	92.8% за 150 эпох
MNIST · LeNet	99.03% за 85 эпох	99.25% за 12 эпох
CIFAR-100 · ResNet-56	61.6% за 200 эпох	69.0% за 100 эпох

На CIFAR-100 1cycle уже за 25 эпох (65.4%) лучше любого результата со ступенчатым LR.

Табл. 2 статьи, среднее по нескольким запускам. Для MNIST «обычный LR» — политика inv из Caffe

ImageNet: 20 эпох вместо 100

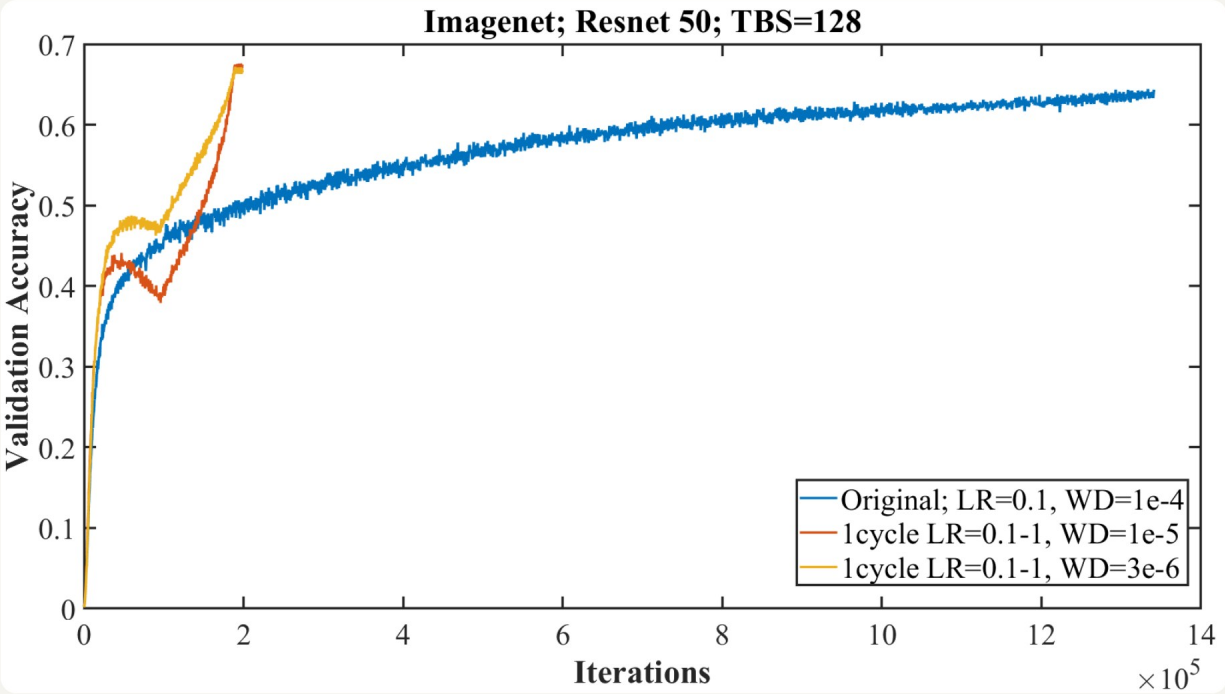


Рис. 6a: ResNet-50 · синяя — обычно, жёлтая и красная — 1cycle

ResNet-50

67.6%

за 20 эпох против 63.7% за 100

Inception-ResNet-v2

74.0%

за 20 эпох против 67.6% за 100

LR 0.05 → 1.0 → 0.00005, weight decay снижен до $3{\cdot}10^{-6}$ – 10^{-5}

Другие наблюдения

Глубина сети

ResNet-20: 90.4% против 88.6%

ResNet-110: 92.1% против 91.0%

Выигрыш больше у неглубоких сетей

Размер батча

Больший батч даёт немного лучшую точность

Разрыв train/test почти одинаковый

Большой батч позволяет больший LR

Другие методы

Адаптивные методы сами большой LR не находят

С 1cycle: Nesterov, AdaGrad, AdaDelta — да, Adam — нет

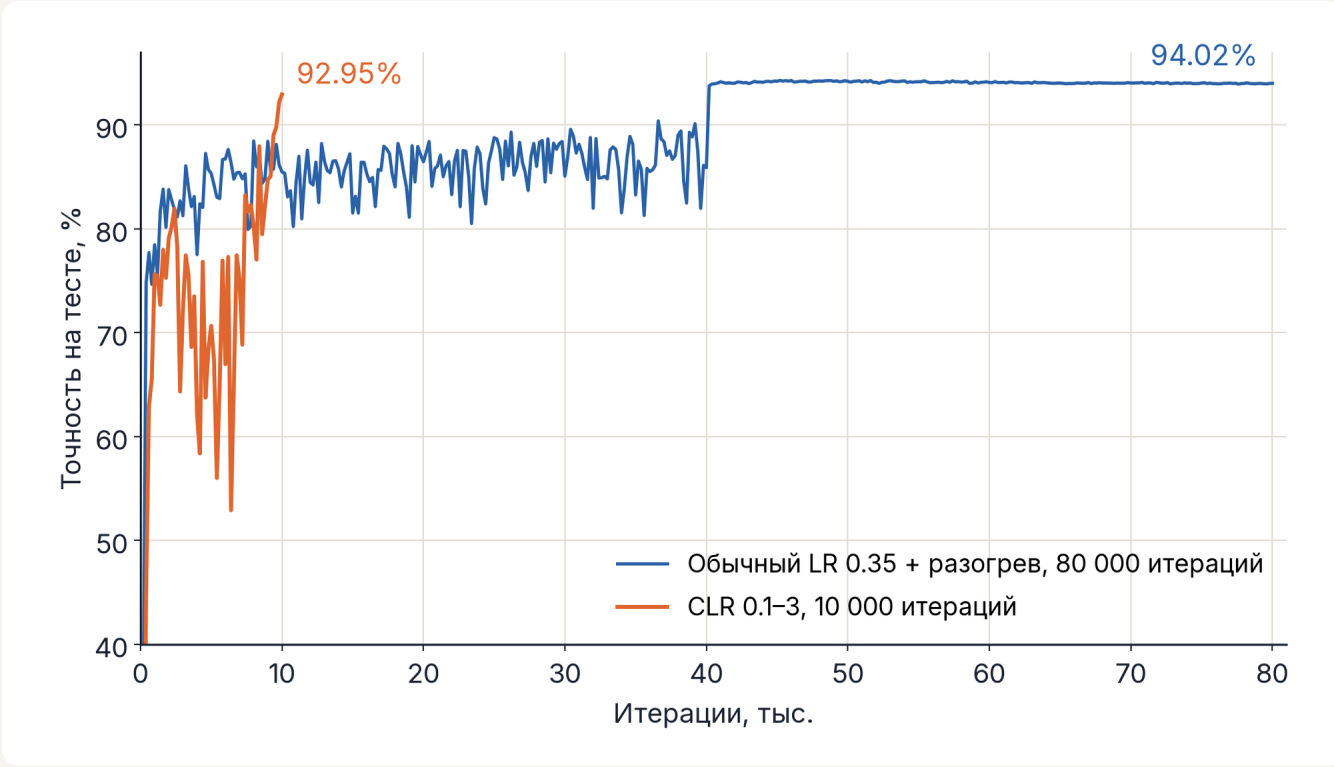
SGDR (косинус с рестартами) — нет

Воспроизведение: главный результат

Тот же эксперимент на PyTorch: ResNet-56, CIFAR-10, батч 1000, одна RTX 5060 Ti

92.5% в статье 92.4%
за 10 000 итераций · CLR, LR 0.1–3

94.0% в статье 91.2%
за 80 000 итераций · обычный LR = 0.35



Мои запуски, аналог рис. 1a. CLR — среднее по 3 запускам, на графике один из них

CLR воспроизвёлся, но обычное обучение оказалось сильнее, чем в статье

Контроль: одинаковое число итераций

Режим	Итераций	Эпох	Точность
CLR 0.1–3	10 000	200	92.5 ± 0.5%
Обычный LR 0.35	10 000	200	92.3 ± 0.4%
CLR 0.1–3	20 000	400	93.1%
Обычный LR 0.35	20 000	400	93.2%
Обычный LR, протокол статьи	80 000	1600	94.0%
He et al., батч 128	64 000	164	92.6%

1600

эпох у бейзлайна статьи — отсюда «в 8 раз быстрее». Стандартному расписанию ResNet хватает 164 эпох

При одинаковом числе итераций CLR и обычный LR дают одну и ту же точность.

± — разброс по 3 запускам. Обычный LR — с разогревом 200 итераций: без него LR 0.35 не обучает сеть вовсе

Когда данных мало: эффект есть, но меньше

Обучающих примеров	Обычный LR 10k итераций	CLR 0.1–3 10k итераций	Разница	Обычный LR 80k итераций
40 000	91.7%	92.7%	+1.0	93.0%
30 000	90.2%	91.9%	+1.7	92.5%
20 000	88.1%	90.2%	+2.1	90.9%
10 000	83.8%	86.2%	+2.5	87.4%

+2.5

процентного пункта при 10 000 примеров — а в статье +9.2

При равном бюджете выигрыш CLR растёт, когда данных меньше, — как в статье. Но обычный LR на 80 000 итераций точнее CLR на всех выборках.

Мои запуски: ResNet-56, CIFAR-10, по одному на конфигурацию

Что ещё проверил

LR range test

Точность 70–85% вплоть до LR = 3, как в статье

Train loss с ростом LR растёт, а test loss не падает

Регуляризация видна лишь наполовину

Weight decay

1e-3: сеть «умирает», точность 10%

1e-4…1e-6: обучение идёт нормально

С BatchNorm WD повышает эффективный LR

Верхняя граница LR

Лучшая — 2.5 (в статье 3)

От 1.5 до 3.5 разница ~1 п.п.

Очень большой LR не обязателен

Выводы и критический взгляд

Что я вынес

- 1cycle воспроизводится: 92.5% за 200 эпох, как в статье
- Большой LR — регуляризация: weight decay при нём нужно ослаблять
- На малых данных выигрыш есть, но до 2.5 п.п., а не 9.2
- 1cycle стал стандартом: OneCycleLR в PyTorch, fit_one_cycle в fast.ai

Что можно покритиковать

- Слабый бейзлайн: 91.2% против 93.03% у He et al.; у меня 94.0%
- «В 8 раз быстрее» — против 1600 эпох; при равном бюджете разницы нет
- Скорость считают в итерациях, а не во времени; батч 1000 на 8 GPU
- Оценку LR через гессиан авторы не проверяли; ImageNet — 2 запуска

Спасибо за внимание!

Вопросы?

Статья: L. N. Smith, N. Topin. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates · arxiv.org/abs/1708.07120